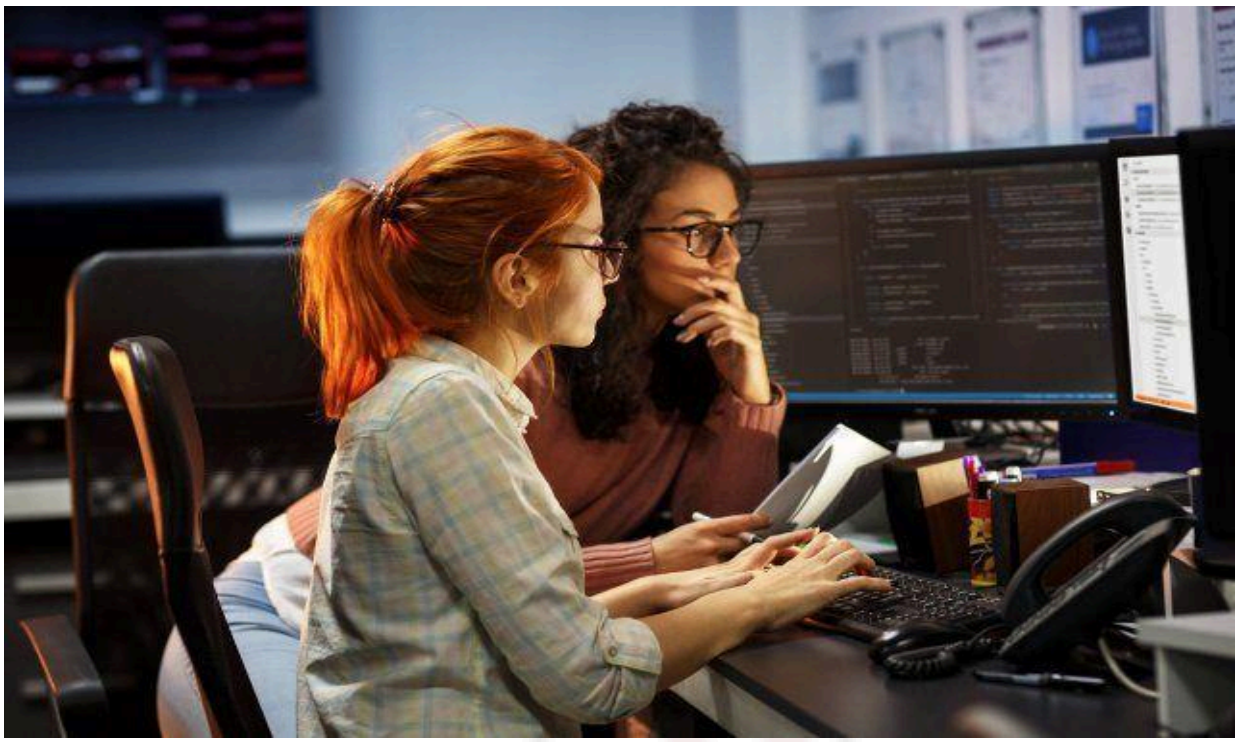

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Programación Orientada a Objetos	Unidad #: 1	Tema: Programación y Principios de Diseño Orientado a Objetos.
	SEMANA 2	Autor:

Introducción:

Bienvenidos al material de lectura #02 de la Unidad #1 de la materia Lógica de Programación. En este documento exploramos sobre la representación visual mediante diagramas de flujo y validación de lógica manual.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en las actividades propuestas, ya que contribuirán significativamente a tu proceso de aprendizaje.

1.4. Representación visual: Diagramas de Flujo Modernos

Objetivo: Comprender que la programación no empieza escribiendo código, sino diseñando "planos" visuales que cualquier miembro del equipo (técnico o no técnico) pueda entender.

A. El Plano del Ingeniero de Software

Así como un arquitecto no comienza a construir una casa apilando ladrillos sin antes dibujar un plano, un ingeniero de software no escribe código sin antes trazar la lógica. Los **diagramas de flujo** son la representación gráfica de un algoritmo.

Utilizan formas geométricas estándar regidas por normativas internacionales (como ANSI/ISO) para asegurar que un diagrama hecho en Japón sea perfectamente entendible por un programador en México.

- **Óvalo (Terminal):** Indica el `INICIO` o `FIN` del algoritmo.
- **Paralelogramo (Entrada/Salida):** Representa la lectura de datos (ej. usuario tecleando su contraseña) o la impresión de datos en pantalla.
- **Rectángulo (Proceso):** Representa operaciones matemáticas, asignación de variables o transformaciones de datos.
- **Rombo (Decisión):** El motor de la lógica. Evalúa una condición (ej. `¿edad >= 18?`) y bifurca el camino en dos rutas posibles (Sí / No).
- **Flechas (Líneas de Flujo):** Indican el sentido de ejecución. En programación estructurada, el flujo viaja predominantemente de arriba hacia abajo y de izquierda a derecha.

¿Te ha gustado lo que has leído hasta ahora?

Anota a continuación los aspectos que consideras más importantes de esta sección:

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Linea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso

B. Herramientas Modernas de Diagramación

La industria ha evolucionado desde dibujar en papel hasta usar herramientas que transforman texto en diagramas:

1. **PSeInt (Enfoque Educativo):** Es una herramienta fundamental para estudiantes. Permite escribir pseudocódigo en español e inmediatamente autogenera el diagrama de flujo correspondiente. Es excelente para entender la relación directa entre el texto lógico y la gráfica.
2. **Lucidchart / Draw.io (Enfoque Visual Tradicional):** Herramientas de lienzo libre tipo "arrastrar y soltar" (Drag & Drop). Son el estándar en oficinas corporativas para diseñar arquitecturas de sistemas completos o flujos de experiencia de usuario (UX).
3. **Mermaid.js (Enfoque "Diagramas como Código"):** Es la tendencia actual en Ingeniería de Software. En lugar de dibujar, el programador escribe texto usando Markdown y la herramienta genera el gráfico.
    - *Ventaja:* Al ser texto, estos diagramas se pueden guardar en **Git** (control de versiones), permitiendo ver quién modificó el diagrama y cuándo.

1.5. El Ciclo de un Algoritmo: El Modelo IPO

Objetivo: Internalizar que todo sistema informático, desde una simple calculadora hasta el motor de búsqueda de Google, obedece al mismo principio físico y lógico: Entrada, Proceso y Salida.

El modelo **IPO (Input, Process, Output)** es la columna vertebral del diseño de software. Si un estudiante no puede identificar cuáles son las entradas, qué debe procesarse y cuál es la salida esperada, no podrá programar la solución.

Fase 1: Entrada (Input)

Es la recolección de los datos "crudos" necesarios para que el algoritmo funcione. Estos datos provienen del exterior hacia la memoria (RAM).

- *Fuentes comunes:* El teclado del usuario, el clic de un mouse, la lectura de un archivo `.json`, la lectura de un sensor de temperatura, o una petición a través de internet (API).
- *Pregunta clave:* ¿Qué ingredientes necesito para resolver este problema?

Fase 2: Procesamiento (Process)

Es el "cerebro" de la operación (ejecutado por la CPU). Aquí es donde los datos crudos se transforman en información útil mediante la lógica y las matemáticas.

- *Acciones comunes:* Realizar cálculos (`total = precio * cantidad`), ordenar listas de la A a la Z, filtrar información (bifurcaciones con sentencias `Si/Entonces`), o repetir tareas (Ciclos `Mientras`).
- *Pregunta clave:* ¿Qué reglas de negocio o fórmulas matemáticas debo aplicar a mis entradas?

Fase 3: Salida (Output)

Es la entrega del resultado final. El sistema envía la información procesada desde la memoria hacia un medio perceptible u otro sistema.

- *Destinos comunes:* Imprimir texto en el monitor, emitir un sonido, encender un motor, o guardar la información final en una base de datos o archivo `.csv`.

- *Pregunta clave:* ¿Cuál es el resultado que el usuario o el cliente está esperando ver?

Ejemplo Ingenieril (Validación de Login):

- **Input:** Usuario (`juan@mail.com`) y Contraseña (`1234`).
- **Process:** Buscar en la base de datos si existe el usuario. Si existe, encriptar la contraseña introducida y compararla con la guardada.
- **Output:** Mensaje en pantalla: *"Bienvenido al sistema"* o *"Credenciales inválidas"*.

1.6. Pruebas de Escritorio y Validación Lógica Manual

Objetivo: Desarrollar la capacidad de simular la ejecución de un programa en la mente y en papel, previniendo errores críticos antes de la fase de codificación.

A. ¿Qué es una Prueba de Escritorio (Trace Table)?

Una prueba de escritorio es la ejecución manual de un algoritmo. Consiste en que el programador "juegue a ser la CPU y la memoria RAM", leyendo el pseudocódigo línea por línea y anotando cómo cambian los valores de las variables en cada paso.

En la industria del software, corregir un error lógico durante la fase de diseño (papel) cuesta centavos; corregirlo cuando el software ya está en producción (Deploy) puede costar millones. La prueba de escritorio es la primera línea de defensa contra los *Bugs*.

B. Metodología de Validación (Paso a Paso)

Para realizar una prueba rigurosa, se utiliza una matriz o tabla de seguimiento:

1. **Columnas:** Se crea una columna para cada variable del algoritmo, además de una columna final para registrar la "Salida en pantalla" o acciones.
2. **Filas:** Cada fila representa una instrucción ejecutada o un ciclo iterado.
3. **Ejecución:** Se sigue el flujo del algoritmo estrictamente. Si el algoritmo dice que sumes 1, sumas 1, sin asumir nada que no esté escrito.

C. Ejemplo Práctico: El Algoritmo del Contador

Pseudocódigo propuesto:

```
1. INICIO
2. DEFINIR contador = 0
3. MIENTRAS contador < 3 HACER:
4.     contador = contador + 1
5.     IMPRIMIR "El número es: ", contador
6. FIN MIENTRAS
7. IMPRIMIR "Terminado"
8. FIN
```

La Prueba de Escritorio (Trace Table):

Paso / Línea ejecutada	Variable: `contador`	Condición: `contador < 3`	Salida en Pantalla (Output)
1-2. Inicialización	**0**	-	-
3. Evaluación Ciclo 1	0	**Verdadero** (0 es menor que 3)	-
4-5. Proceso Ciclo 1	**1**	-	"El número es: 1"
3. Evaluación Ciclo 2	1	**Verdadero** (1 es menor que 3)	-
4-5. Proceso Ciclo 2	**2**	-	"El número es: 2"
3. Evaluación Ciclo 3	2	**Verdadero** (2 es menor que 3)	-
4-5. Proceso Ciclo 3	**3**	-	"El número es: 3"
3. Evaluación Ciclo 4	3	**Falso** (3 NO es menor que 3)	*(Se rompe el ciclo, salta a línea 7)*
7. Fin del programa	3	-	"Terminado"

Conclusión de la prueba: El algoritmo funciona correctamente. El ciclo se ejecuta exactamente 3 veces, protegiendo al sistema de un temido "bucle infinito" y las salidas coinciden con lo esperado.

1. ¿Qué es PSeInt y por qué es vital en la Ingeniería?

PSeInt (Pseudo Intérprete) es una herramienta educativa diseñada para asistir a los estudiantes en sus primeros pasos en la programación. En lugar de forzar al alumno a memorizar la sintaxis estricta y en inglés de un lenguaje real (como Python o C++), PSeInt permite escribir algoritmos en **pseudocódigo en español**.

En la ingeniería de software, el mayor reto inicial no es aprender un lenguaje, sino aprender a **estructurar la mente**. PSeInt elimina la "fatiga de sintaxis" (errores por olvidar un punto y coma o una llave) y permite que el cerebro se concentre al 100% en la resolución lógica del problema.

Características Clave:

- **Intérprete en Español:** Permite dar órdenes a la computadora en el idioma nativo del estudiante.
- **Generador de Diagramas de Flujo:** Convierte automáticamente el texto escrito en un diagrama visual (y viceversa).
- **Ejecución Paso a Paso:** Funciona como un depurador visual (*debugger*), mostrando cómo fluye la información línea por línea, simulando una prueba de escritorio automatizada.

---

2. Anatomía de un Algoritmo en PSeInt

Todo programa en PSeInt sigue una estructura básica obligatoria. Es el "cascarón" donde vivirá nuestra lógica.

```
Algoritmo NombreDelPrograma

    // Aquí va la lógica (Declaraciones, Entradas, Procesos y Salidas)

FinAlgoritmo
```

Nota: Los comentarios (anotaciones para humanos que la máquina ignora) se escriben usando dos barras diagonales `//`.

---

3. El Manejo de Memoria: Variables y Tipos de Datos

Antes de procesar datos, debemos decirle a la computadora qué espacio de memoria (RAM) necesitamos reservar. En PSeInt, esto se hace con la palabra reservada `Definir`.

Existen cuatro tipos de datos fundamentales:

1. **Entero:** Números sin decimales (ej. Edad, cantidad de alumnos).
2. **Real:** Números con decimales (ej. Precio, promedio de notas).
3. **Caracter / Texto:** Letras, palabras o frases (ej. Nombre, dirección).
4. **Logico:** Solo puede guardar dos estados: `Verdadero` o `Falso` (Usado para banderas o *flags* lógicas).

Ejemplo de declaración:

```
Definir edad Como Entero

Definir estatura Como Real

Definir nombre Como Caracter
```

---

4. El Modelo IPO en PSeInt (Entrada / Salida / Asignación)

A. Salida (Output) - El comando `Escribir`

Sirve para mostrar mensajes o resultados en la pantalla del usuario.

- *Sintaxis:* `Escribir "Hola, bienvenido al sistema"`
- *Para mostrar variables:* `Escribir "Tu edad es: ", edad`

B. Entrada (Input) - El comando `Leer`

Pausa la ejecución del programa, espera a que el usuario teclee un valor y lo guarda en una variable previamente definida.

- *Sintaxis:* `Leer edad`

C. Procesamiento (Asignación) - El operador `<-`

En lógica pura, la asignación no usa el símbolo de igual (`=`), sino una flecha (`<-`). Esto significa "toma el valor de la derecha y guárdalo en la variable de la izquierda".

- *Sintaxis:* `promedio <- (nota1 + nota2) / 2`
- *Incrementos:* `contador <- contador + 1`

5. Estructuras de Control (El Motor de la Lógica)

Las computadoras serían inútiles si solo ejecutaran instrucciones en línea recta. Las estructuras de control permiten bifurcar caminos o repetir acciones.

A. Condicionales (Toma de Decisiones): `Si-Entonces`

Evalúa una condición lógica. Si es verdadera, ejecuta un bloque de código; si es falsa, ejecuta otro.

```
Si edad >= 18 Entonces

    Escribir "Acceso permitido. Eres mayor de edad."

SiNo

    Escribir "Acceso denegado."

FinSi
```

B. Ciclos (Repetición): `Mientras` (While Loop)

Permite repetir un bloque de código **mientras** una condición siga siendo verdadera. Es vital para validaciones y evitar que el programa termine prematuramente.

```
Definir pin Como Entero

Escribir "Ingrese su PIN de 4 dígitos:"

Leer pin

Mientras pin <> 1234 Hacer  // El símbolo <> significa
"Diferente de"

    Escribir "PIN incorrecto, intente de nuevo:"

    Leer pin

FinMientras

Escribir "Acceso concedido"
```

6. La Herramienta "Paso a Paso" (Debugging)

Una de las funciones más poderosas de PSeInt para un ingeniero en formación es el botón de **"Ejecutar paso a paso"** (representado por unos pies en la barra de herramientas).

Al usarlo, PSeInt ejecuta el código lentamente, resaltando en verde la línea exacta que está leyendo la CPU, y muestra una ventana emergente donde se ve cómo cambian los valores de las variables en tiempo real. Es la versión digital e interactiva de la **Prueba de Escritorio**.